



Network Programming - 17

Queue 자료구조와 네트워크 버퍼

afewhee@gmail.com





- Queue 자료구조 기반 Buffer
- 선형 큐 (Linear Queue)
- 원형 큐 (Circular Queue) 통한 Buffer 재사용
- 문자열 큐 (String Queue) 통한 데이터 누적 구조
- Network 송수신 Buffer
- Packet 처리 구조 기반





- 선입 선출
 - ◆ FIFO (First In First Out)
- 특징
 - ◆ 입력 순서 보존
 - ◆ 처리 순서 보장
 - ◆ 대기열 구조
- 동기화





- 동기화 관련된 모든 것
- 윈도우 메시지 처리
 - ◆ 키보드 입력 버퍼
 - ◆ 프린터 출력 대기열
- 게임 이벤트 큐
- 네트워크 송신 대기열
- 네트워크 수신 버퍼



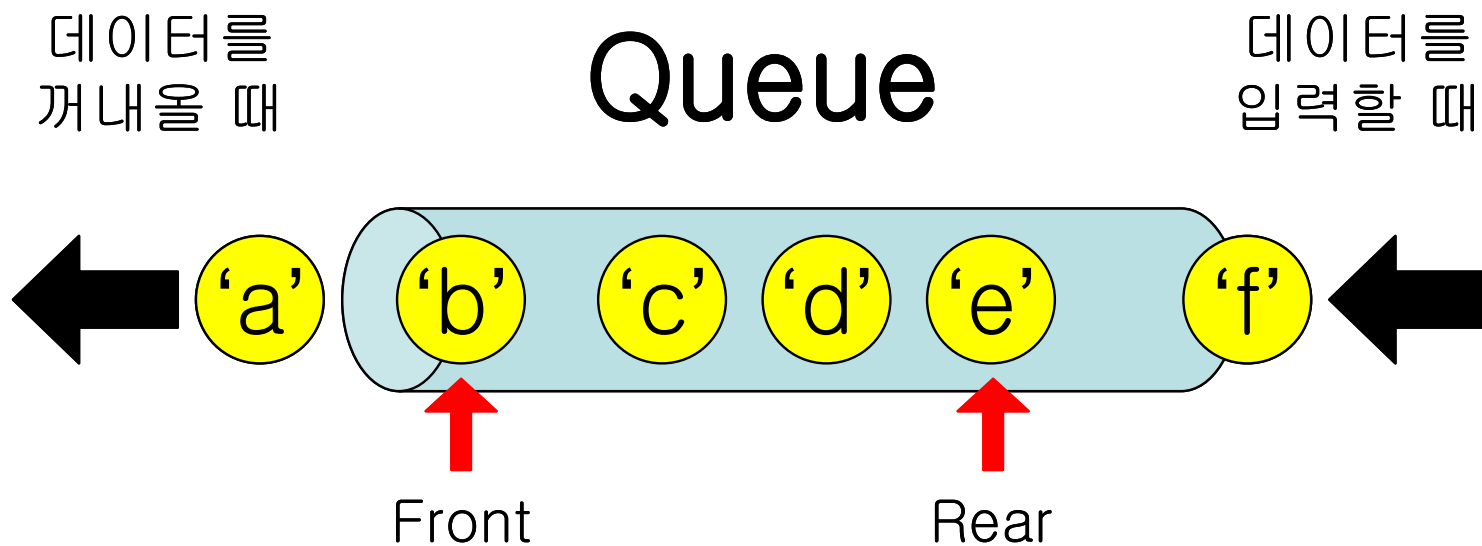


- Enqueue
 - ◆ Queue 뒤쪽 데이터 추가
- Dequeue
 - ◆ Queue 앞쪽 데이터 제거
- Rear
 - ◆ 다음 입력 위치
- Front
 - ◆ 다음 출력 위치





- Queue





- Linear Queue
 - ◆ 배열 기반 Queue
 - ◆ 데이터 뒤쪽 추가
 - ◆ 데이터 앞쪽 제거
- Rear
 - ◆ 현재 저장 개수
 - ◆ 다음 입력 위치





17.2.2 Linear Queue 구현

```
class CQueueLinear           // 큐 클래스
{
protected:
    int* m_pBuf   {};        // 큐의 버퍼
    int  m_iRear   {};        // 입력 문자열 끝 위치
    int  m_iWidth  {};        // 큐의 버퍼 크기

public:
    CQueueLinear();
    CQueueLinear(int iWidth);
    virtual ~CQueueLinear();

    int Enqueue(int iIn);      // 데이터 추가
    int Dequeue(int* iOut);    // 데이터 꺼냄
};
```





- m_pBuf
 - ◆ Queue 저장 공간
- m_iRear
 - ◆ 현재 데이터 개수
 - ◆ 다음 입력 위치
- m_iWidth
 - ◆ Queue 최대 크기





- 동적 배열 생성
- Queue 크기 설정
- 버퍼 0 초기화

```
CQueLinear::CQueLinear(int iWidth)
{
    m_iRear = 0;
    m_iWidth= iWidth;

    m_pBuf  = new int[m_iWidth+1]{}; // End Pointer 하나 더 추가...
}
```





- 공간 확인
- Rear 위치 데이터 저장
- Rear 증가

```
int CQueLinear::Enqueue(int iIn) // 큐에 데이터 추가
{
    if( m_iRear == m_iWidth)      // 남아 있는 공간이 없으면 채우지 못함
        return -1;                // Failed

    m_pBuf[m_iRear++] = iIn;
    return 0;
}
```



- 0번 데이터 출력
- 남은 데이터 앞으로 이동
- Rear 감소

```
int CQueLinear::Dequeue(int* iOut)           // 큐에서 데이터 가져오기
{
    if(0 == m_iRear)                          // 버퍼가 비워져 있는 경우
        return -1;                          // Failed

    *iOut = m_pBuf[0];

    int i=0;
    for(; i < m_iRear; ++i)
        *(m_pBuf+i) = *(m_pBuf+i+1);        // 한 단계씩 앞 자리로 이동

    *(m_pBuf+i) = 0;                          // 마지막 자리 0으로 초기화
    --m_iRear;                                // Rear 위치 이동
    return 0;
}
```



Initial

```
[ ][ ][ ][ ][ ]  
rear = 0
```

Enqueue A

```
[A][ ][ ][ ][ ]  
rear = 1
```

Enqueue B

```
[A][B][ ][ ][ ]  
rear = 2
```

Dequeue

```
[B][ ][ ][ ][ ]  
rear = 1
```





- Queue 크기 검사
- CQueueLinear 객체 생성
- 응용 클래스 내부 보관

```
int CQueueApp::Init(int iQueueSize /* = MAX_QUEUE */)
{
    if(iQueueSize<0)                // 큐의 크기가 작으면 실패
        return -1;

    m_pQueue = new CQueueLinear(iQueueSize);
    return 0;
}
```





```
void CQueueApp::EnqueuingMode()
{
    int iSize = strlen(sMsg);
    int iRet = m_pQueue->Enqueue(sMsg[0]);

    if(FAILED(iRet))
    {
        char* p = sMsg;
        p += iSize - iRet;
        printf("\n\n 큐의 공간이 부족해 전부 채워지 못했습니다.\n");
        printf(" 남은 데이터: %s\n", p);
    }
}
```





17.2.10 Linear Queue DeQueue 예

```
void CQueueApp::DequeuingMode()
{
    int iVal= 0;
    int hr  = m_pQueue->Dequeue(&iVal);

    if(FAILED(hr))
    {
        printf("Queue가 비어 있습니다.\n\n");
        continue;
    }

    printf("\n\n    가져온 글자: %c\n", iVal);
}
```





- Dequeue 발생
 - ◆ 남은 데이터 전체 이동
- 데이터 개수 증가
 - ◆ 복사 비용 증가
- Queue 사용 빈도 증가
 - ◆ 성능 낭비 증가





- Linear Queue 문제
 - ◆ Dequeue 마다 데이터 이동
- Circular Queue 해결
 - ◆ 데이터 이동 제거
 - ◆ Front Rear 위치 이동
- 배열 재사용
 - ◆ 끝 도달 후 처음 위치 이동





- Write 위치 분리
- Read 위치 분리
- 단일 생산자 / 단일 소비자 구조
- Lock 비용 감소 가능





- Circular Queue
 - ◆ 배열 원형 사용
 - ◆ Front 출력 위치
 - ◆ Rear 입력 위치
 - ◆ Space 남은 공간
- 다음 위치 계산
 - ◆ $\text{index} = \text{index} \% \text{width}$





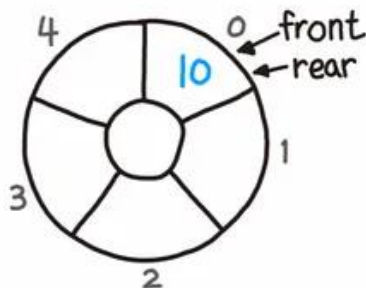
- 크기 5 Queue
 - ◆ $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$
- Rear 증가
 - ◆ 끝 도달
 - ◆ 처음 위치 이동
- Front 증가
 - ◆ 끝 도달
 - ◆ 처음 위치 이동



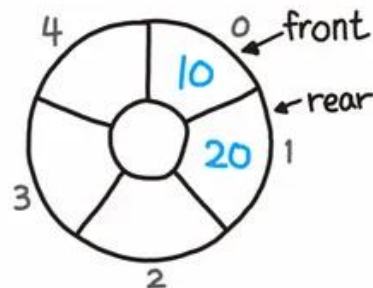


17.3.5 Circular Queue 동작 예

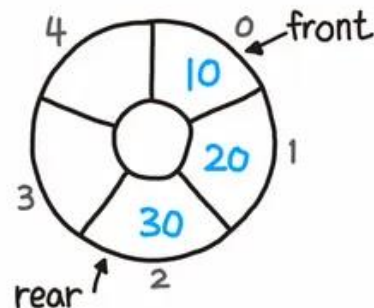
enqueue(10)



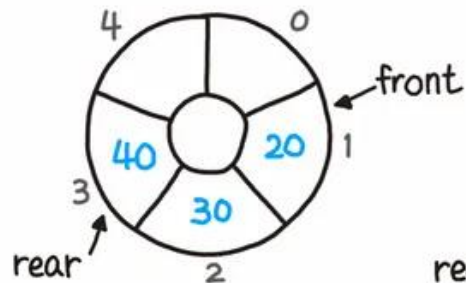
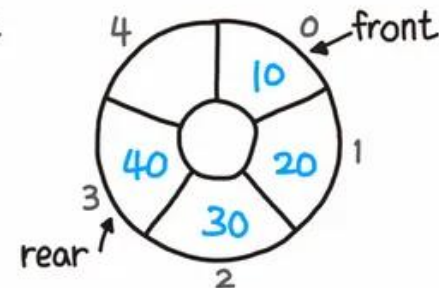
enqueue(20)



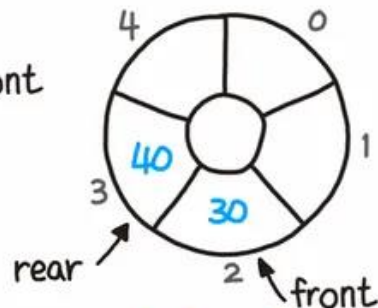
enqueue(30)



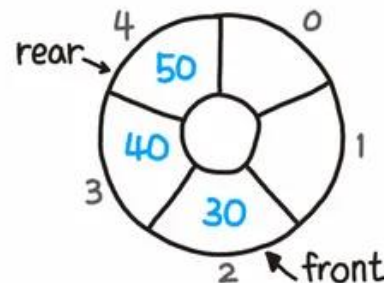
enqueue(40)



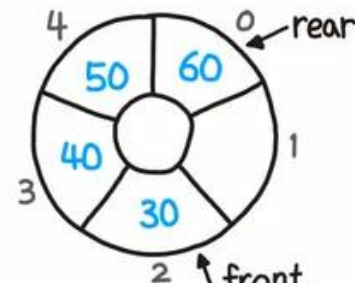
dequeue()



dequeue()



enqueue(50)



enqueue(60)





17.3.6 Circular Queue 구현

```
class CQueCircular
{
protected:
    int*    m_pBuf        {};           // 큐의 버퍼
    int     m_iFront      {};           // 시작 위치
    int     m_iRear       {};           // 끝 위치
    int     m_iSpace      {};           // 남아있는 큐의 버퍼 크기
    int     m_iWidth      {};           // 큐의 버퍼 크기

public:
    CQueCircular();
    CQueCircular(int iWidth);
    virtual ~CQueCircular();

    int      Enqueue(int iIn);          // 데이터 추가
    int      Dequeue(int* iOut);        // 데이터 꺼냄
};
```





- **m_iFront**
 - ◆ 다음 출력 위치
- **m_iRear**
 - ◆ 다음 입력 위치
- **m_iSpace**
 - ◆ 남은 공간





- Front 0
- Rear 0
- Space 전체 크기
- 동적 배열 생성

```
CQueCircular::CQueCircular(int iWidth)
{
    m_iFront    = 0;
    m_iRear     = 0;
    m_iSpace    = iWidth;
    m_iWidth    = iWidth;

    m_pBuf = new int[m_iWidth+1]{}; // End Pointer 하나 더 추가...
}
```





- 남은 공간 확인
- Rear 위치 저장
- Space 감소
- Rear 순환 이동

```
int CQueCircular::Enqueue(int iIn) // 큐에 데이터 추가
{
    if( 0 == m_iSpace)             // 남아 있는 공간이 없으면 채우지 못함
        return -1;                // Failed

    m_pBuf[m_iRear] = iIn;

    --m_iSpace;                    // 빈 영역 감소

    ++m_iRear;
    m_iRear%= m_iWidth;            // Rear 위치 이동
    return 0;
}
```





- 비어 있음 확인
- Front 위치 출력
- Space 증가
- Front 순환 이동

```
int CQueCircular::Dequeue(int* iOut) // 큐에서 데이터 가져오기
{
    if(m_iSpace == m_iWidth)         // 버퍼가 비워져 있는 경우
        return -1;                   // Failed

    *iOut = m_pBuf[m_iFront];

    ++m_iSpace;                       // 빈 영역 증가

    ++m_iFront;
    m_iFront%= m_iWidth;              // Front 위치 이동

    return 0;
}
```





- Linear Queue
 - ◆ 구현 단순
 - ◆ Dequeue 시 데이터 이동 → 성능 손실 가능
- Circular Queue
 - ◆ 데이터 이동 없음 → 버퍼 재사용 효율 높음
 - ◆ Front Rear 관리 필요





● 네트워크 소켓 이벤트의 사용자 정의 메시지 큐잉

```
struct NwMsg {  
    unsigned int socket;  
    unsigned int nMsg;  
};  
  
class CQueCircular {  
protected:  
    NwMsg*   m_pBuf    {};  
    int      m_iFront  {};  
    int      m_iRear   {};  
    int      m_iSpace  {};  
    int      m_iWidth  {};  
  
public:  
    CQueCircular();  
    CQueCircular(int iWidth);  
    virtual ~CQueCircular();  
  
    int      Enqueue(NwMsg* pIn);  
    int      Dequeue(NwMsg* pOut);  
};
```





- NwMsg 단위 저장
- 메시지 전체 복사

```
int CQueCircular::Enqueue(NwMsg* pIn)    // 큐에 데이터 추가
{
    if( 0 == m_iSpace)                  // 남아 있는 공간이 없으면 채우지 못함
        return -1;                      // Failed

    memcpy(&m_pBuf[m_iRear], pIn, sizeof(NwMsg));

    --m_iSpace;                          // 빈 영역 감소

    ++m_iRear;
    m_iRear%= m_iWidth;                  // Rear 위치 이동
    return 0;
}
```





- NwMsg 단위 출력
- Front 위치 데이터 복사
- Front 순환 이동

```
int CQueCircular::Dequeue(NwMsg* pOut) // 큐에서 데이터 가져오기
{
    if(m_iSpace == m_iWidth)           // 버퍼가 비워져 있는 경우
        return -1;                     // Failed

    memcpy(pOut, &m_pBuf[m_iFront], sizeof(NwMsg));

    ++m_iSpace;                         // 빈 영역 증가

    ++m_iFront;
    m_iFront%= m_iWidth;               // Front 위치 이동

    return 0;
}
```





- 네트워크 데이터
 - ◆ 문자열 형태 메시지
 - ◆ 바이트 배열 형태 수신
- 정수 Queue 부족
 - ◆ 문자열 저장 필요
 - ◆ 연속 데이터 저장 필요





17.5.2 String Queue 클래스

```
class CQueStr
{
protected:
    char*    m_sBuf    {};           // 큐의 버퍼
    int      m_iRear    {};           // 입력 문자열 끝 위치
    int      m_iWidth   {};           // 큐의 버퍼 크기

public:
    CQueStr();
    CQueStr(int iWidth);
    virtual ~CQueStr();

    int      Enqueue(char* sIn,int iSize=1);
    int      Dequeue(char*sOut,int iSize=1);

    void      Clear();                // 큐의 내용을 지움
    int      GetWidth();              // 큐의 크기를 가져옴
    int      GetRemain();              // 큐의 남아 있는 크기를 가져옴
    void      Resize(int iSize);       // 큐의 사이즈를 다시 설정
};
```





- 문자 버퍼 생성
- 문자열 끝 공간 추가
- 0 초기화

```
CQueStr::CQueStr(int iWidth)
{
    m_iRear = 0;
    m_iWidth= iWidth;

    // End Pointer 하나 더 추가
    m_sBuf  = new char[m_iWidth+1]{};
}
```





17.5.4 String Queue Enqueue

- 입력 크기 확인
- 남은 공간 계산
- 저장 가능 크기만 복사
- 남은 데이터 크기 반환

```
int CQueStr::Enqueue(char* sIn,int iSize/*=1*/) // 큐에 데이터 추가
{
    if(0== iSize)
        return 0;
    if( m_iRear == m_iWidth)                // 남아 있는 공간이 없으면 채우지 못함
        return iSize;
    int iRet =0;                            // 반환 값
    int iRemain = m_iWidth - m_iRear;      // 남은 공간 계산
    if(iSize>iRemain)                       // 반환 값 설정
    {
        iRet  = iSize-iRemain;
        iSize = iRemain;
    }
    for(int i=0; i<iSize; ++i)
        *(m_sBuf + m_iRear++) = *(sIn+i); // 데이터 복사, Rear 위치 이동
    return iRet;
}
```





17.5.5 String Queue Dequeue

- 요청 크기만 출력
- 출력 데이터 0 초기화
- 남은 데이터 앞으로 이동
- 출력 길이 반환

```
int CQueStr::Dequeue(char* sOut,int iSize/*=1*/)    // 큐에서 데이터 가져오기
{
    if(0 == m_iRear)                                // 버퍼가 비워져 있는 경우
        return 0;
    if(iSize>m_iRear)                                // 만약 요청한 크기가 저장되어 있는
        iSize = m_iRear;                            // 크기보다 크다면 재조정
    int i=0;
    for(; i<iSize; ++i) {
        *(sOut+i) = *(m_sBuf + i);                  // 데이터 복사
        *(m_sBuf+i) =0;                             // 복사되면 큐의 내용을 0으로 초기화
    }
    if(iSize != m_iRear) {                            // 일부만 복사했다면 그 만큼만 이동
        for(i=iSize; i < m_iRear; ++i)
            *(m_sBuf+i-iSize) = *(m_sBuf+i);        // 한 단계씩 앞 자리로 이동

        *(m_sBuf+i-iSize) = 0;                       // 마지막 자리도 0으로 초기화
    }
    m_iRear -= iSize;                                // Rear 위치 이동
    return iSize;                                    // 가져온 문장의 길이를 반환
}
```





17.5.6 String Queue 사용 예

```
int CQueueApp::Init(int iQueueSize /* = MAX_QUEUE */)
{
    m_pQueue = new CQueueStr(iQueueSize);
    return 0;
}
```

```
void CQueueApp::EnqueuingMode()
{
    char sMsg[256]={0};

    int iSize = strlen(sMsg);
    int iRet = m_pQueue->Enqueue(sMsg, iSize);
}
```

```
void CQueueApp::DequeuingMode()
{
    char    sMsg[256]={0};
    iSize = m_pQueue->Dequeue(sMsg, iSize);
    sMsg[iSize]=0;
}
```





- char Queue
 - ◆ 문자 단위 저장
- uint8_t Queue
 - ◆ 바이트 단위 저장
- Network Buffer
 - ◆ 수신 바이트 저장





- `uint8_t` 버퍼 사용
- Front Rear 사용
- Space 사용
- Circular Queue 구조 적용





17.6.3 BYTE 기반 String Queue 클래스

```
#include <stdint.h>

class CQueStr {
protected:
    uint8_t*    m_sBuf        {};           // 큐의 버퍼
    int         m_iFront      {};           // Front 위치
    int         m_iRear       {};           // Rear 위치위치
    int         m_iSpace      {};           // 남아 있는 공간
    int         m_iWidth      {};           // 큐의 버퍼 크기

public:
    CQueStr();
    CQueStr(int iWidth);
    virtual ~CQueStr();
    int      Enqueue(void* sIn,int iSize);   // 데이터 추가
    int      Dequeue(void*sOut,int* iSize);  // 데이터 가져오기
    void      Clear();                      // 큐의 내용을 지움
    int      GetWidth();                    // 큐의 크기를 가져옴
    int      GetRemain();                   // 큐의 남아 있는 크기를 가져옴
    void      Resize(int iSize);            // 큐의 사이즈를 다시 설정
};
```





17.6.4 BYTE Queue Enqueue

- void* 입력
- BYTE* 변환
- 바이트 단위 복사
- Rear 순환 이동

```
int CQueStr::Enqueue(void* sIn,int iSize/*=1*/) // 큐에 데이터 추가
{
    if( 2>= m_iSpace || iSize>m_iSpace) // 남아 있는 공간이 없거나 입력 사이즈보다 작으면
        return -1; // 실패
    auto pIn = (uint8_t*)sIn;
    for(int i=0; i<iSize; ++i)
    {
        *(m_sBuf + m_iRear) = *(pIn+i); // 데이터 복사, Rear 위치 이동

        ++m_iRear;
        m_iRear %=m_iWidth;
    }
    m_iSpace -= iSize;
    return 0;
}
```





17.6.5 BYTE Queue Dequeue

- Front 위치 크기 확인
- BYTE 단위 출력
- Front 순환 이동
- 출력 크기 반환

```
int CQueStr::Dequeue(void* sOut, int* pSize /*=1*/) // 큐에서 데이터 가져오기
{
    if(m_iSpace >= (m_iWidth-2) ) // 버퍼가 비워져 있는 경우
        return -1;
    int iSize = m_sBuf[m_iFront]; // Front의 위치는 길이
    auto pOut = (uint8_t*)sOut;
    for(int i=0; i<iSize; ++i)
    {
        *(pOut+i) = *(m_sBuf + m_iFront); // 데이터 복사
        *(m_sBuf + m_iFront) = 0; // 복사되면 큐의 내용을 0으로 초기화

        ++m_iFront;
        m_iFront%=m_iWidth; // Front위치 이동
    }
    m_iSpace += iSize;
    *pSize = iSize; // 가져온 문장의 길이를 반환
    return 0;
}
```





- 송신 데이터
 - ◆ "HELLO"
- 수신 가능 형태 1
 - ◆ "HELLO"
- 수신 가능 형태 2
 - ◆ "HE"
 - ◆ "LLO"
- 수신 가능 형태 3
 - ◆ "HELLOWORLD"





- 송신 Queue
 - ◆ 보낼 데이터 임시 저장
 - ◆ 순서 보장
- 수신 Queue
 - ◆ 받은 데이터 임시 저장
 - ◆ 처리 순서 보장
- 패킷 Queue
 - ◆ 완성 메시지 저장
 - ◆ 게임 로직 전달





- Client
 - ◆ Network
 - ◆ Buffer
 - ◆ Packet
 - ◆ Game Logic
- Network Thread
 - ◆ 데이터 수신
 - ◆ Buffer 저장
 - ◆ Packet 분리
 - ◆ Queue 전달





- Queue
 - ◆ FIFO 구조
 - ◆ 입력 순서 보존
- Linear Queue
 - ◆ 구현 단순
 - ◆ Dequeue 데이터 이동
- Circular Queue
 - ◆ 데이터 이동 제거
 - ◆ Front Rear 순환
- String Queue
 - ◆ 문자열 저장
 - ◆ Network Buffer 기초

